Dimension 12: Testing, Build System & Store Distribution

Cross-Browser Torrent Extension â€” Complete Testing, Build & Distribution Guide

Research Date: 2025
Sources Consulted: 40+ primary sources including official documentation, GitHub repos, MDN, API docs
Confidence Level: High

Table of Contents

---

1. Executive Summary

This document provides a comprehensive, production-ready blueprint for testing, building, and distributing a cross-browser torrent extension. The approach integrates industry best practices from the Boba projectâ€™s testing methodology (pytest, Playwright, CI pipelines) with modern browser extension development tooling.

Key Decisions: - **Build Tool**: WXT (Next-gen Web Extension Framework) built on Vite â€” provides best-in-class DX, auto-reload, cross-browser builds, and automated publishing - **Test Framework**: Jest for unit/integration tests, Playwright for E2E tests - **Mocking**: Manual Jest mocks + sinon-chrome for WebExtension API mocking - **CI/CD**: GitHub Actions with matrix builds across browsers - **Versioning**: release-please with Conventional Commits - **Stores**: Chrome Web Store, Firefox AMO, Edge Add-ons, Opera Add-ons, Yandex

2. Recommended Tech Stack

Category	Tool	Version	Purpose
Build Framework	WXT	^0.20.x	Extension build, dev server, cross-browser packaging
Bundler	Vite	^6.x (via WXT)	Fast ESM-based bundling, HMR
Language	TypeScript	^5.7.x	Type safety, IDE autocomplete
Types	chrome-types	^0.1.x	Chrome API type definitions
Polyfill	webextension-polyfill	^0.12.x	Cross-browser API compatibility
Unit Tests	Jest	^29.x	Unit/integration testing
E2E Tests	Playwright	^1.49.x	Browser automation testing
Linting	ESLint	^9.x	Code quality
TS Plugin	typescript-eslint	^8.x	TypeScript ESLint rules
Formatting	Prettier	^3.4.x	Code formatting
Git Hooks	Husky	^9.x	Git hook management
Staged Lint	lint-staged	^15.x	Run linters on staged files
Coverage	Jest (built-in)	â€”	Istanbul/NYC via Jest
Versioning	release-please	^16.x	Automated semver releases
Firefox Tools	web-ext	^8.x	Build, lint, sign for Firefox
CI/CD	GitHub Actions	â€”	Automated testing & publishing

3. Testing Framework Setup

3.1 Jest Configuration for Extensions

Create `jest.config.ts`:

```
import type { Config } from 'jest';

const config: Config = {
  // Use ts-jest for TypeScript files
  preset: 'ts-jest',

  // Test environment - jsdom for DOM manipulation tests
  testEnvironment: 'jsdom',

  // Root directories to scan for tests
```

```
roots: ['<rootDir>/src', '<rootDir>/tests'],

// Test file patterns
testMatch: [
  '**/__tests__/**/*.ts',
  '**/?(*.)+(spec|test).ts'
],

// Module file extensions
moduleFileExtensions: ['ts', 'tsx', 'js', 'jsx', 'json'],

// Setup files - mock browser APIs before tests run
setupFiles: ['<rootDir>/tests/setup/mock-extension-apis.ts'],

// Setup file after env initialization
setupFilesAfterEnv: ['<rootDir>/tests/setup/jest-setup.ts'],

// Module name mappings for path aliases
moduleNameMapper: {
  '^@/(.*)$': '<rootDir>/src/$1',
  '^@core/(.*)$': '<rootDir>/src/core/$1',
  '^@background/(.*)$': '<rootDir>/src/background/$1',
  '^@content/(.*)$': '<rootDir>/src/content/$1',
  '^@popup/(.*)$': '<rootDir>/src/popup/$1',
  // Mock static assets
  '\\.(css|less|scss|sass)$': 'identity-obj-proxy',
  '\\.(jpg|jpeg|png|gif|svg)$': '<rootDir>/tests/setup/file-mock.ts'
},

// Transform configuration
transform: {
  '^.+\\.tsx?$': ['ts-jest', {
    tsconfig: '<rootDir>/tsconfig.test.json',
    diagnostics: { warnOnly: true }
  }]
},

// Coverage configuration
collectCoverageFrom: [
  'src/**/*.{ts,tsx}',
  '!src/**/*.d.ts',
  '!src/**/index.ts',
  '!src/**/*.config.ts',
  '!src/entrypoints/**' // Entrypoints are tested via E2E
],
coverageThreshold: {
  global: {
    branches: 70,
    functions: 70,
    lines: 70,
    statements: 70
  }
},
coverageReporters: ['text', 'text-summary', 'lcov', 'html'],
```

```
  coverageDirectory: '<rootDir>/coverage',

  // Test timeout
  testTimeout: 10000,

  // Verbose output for CI
  verbose: true,

  // Clear mocks between tests
  clearMocks: true,
  restoreMocks: true,

  // Ignore patterns
  testPathIgnorePatterns: [
    '/node_modules/',
    '/dist/',
    '/.output/',
    '/playwright/'
  ]
};

export default config;
```

3.2 TypeScript Config for Tests

Create `tsconfig.test.json`:

```
{
  "extends": "./tsconfig.json",
  "compilerOptions": {
    "types": ["jest", "node", "chrome"],
    "isolatedModules": false,
    "noEmit": true
  },
  "include": [
    "src/**/*",
    "tests/**/*",
    "**/*.test.ts",
    "**/*.spec.ts"
  ]
}
```

3.3 Package.json Scripts

```
{
  "scripts": {
    "dev": "wxt",
    "dev:firefox": "wxt -b firefox",
    "build": "wxt build",
    "build:firefox": "wxt build -b firefox",
    "build:all": "wxt build -b chrome && wxt build -b firefox && wxt build
    "zip": "wxt zip",
    "zip:all": "wxt zip -b chrome && wxt zip -b firefox && wxt zip -b edge
```

```
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage",
    "test:ci": "jest --ci --coverage --reporters=default --reporters=jest-
    "test:e2e": "playwright test",
    "test:e2e:ui": "playwright test --ui",
    "lint": "eslint src tests --ext .ts,.tsx",
    "lint:fix": "eslint src tests --ext .ts,.tsx --fix",
    "format": "prettier --write \"src/**/*.{ts,tsx,json}\" \"tests/**/*.{t
    "format:check": "prettier --check \"src/**/*.{ts,tsx,json}\" \"tests/*
    "typecheck": "tsc --noEmit",
    "prepare": "husky",
    "release": "release-please"
  }
}
```

3.4 Sample Unit Tests

Background Script Test (`tests/background/magnet-parser.test.ts`**)**

```
import { parseMagnetLink, extractMagnetFromPage } from '@/background/magne

describe('Magnet Parser', () => {
  describe('parseMagnetLink', () => {
    it('should parse a valid magnet link with infohash', () => {
      const magnet = 'magnet:?xt=urn:btih:1234567890abcdef1234567890abcdef
      const result = parseMagnetLink(magnet);

      expect(result).not.toBeNull();
      expect(result?.infoHash).toBe('1234567890abcdef1234567890abcdef12345
      expect(result?.xt).toBe('urn:btih:1234567890abcdef1234567890abcdef12
    });

    it('should parse magnet link with display name', () => {
      const magnet = 'magnet:?xt=urn:btih:1234567890abcdef1234567890abcdef
      const result = parseMagnetLink(magnet);

      expect(result?.displayName).toBe('Example File');
    });

    it('should parse magnet link with trackers', () => {
      const magnet = 'magnet:?xt=urn:btih:1234567890abcdef1234567890abcdef
      const result = parseMagnetLink(magnet);

      expect(result?.trackers).toHaveLength(1);
      expect(result?.trackers[0]).toBe('udp://tracker.example.com:1337');
    });

    it('should return null for invalid magnet link', () => {
      expect(parseMagnetLink('not-a-magnet')).toBeNull();
      expect(parseMagnetLink('')).toBeNull();
      expect(parseMagnetLink('http://example.com')).toBeNull();
    });
```

```
  });

  describe('extractMagnetFromPage', () => {
    it('should extract magnet links from HTML content', () => {
      const html = `
        <html>
          <body>
            <a href="magnet:?xt=urn:btih:abc123">Download 1</a>
            <a href="magnet:?xt=urn:btih:def456">Download 2</a>
            <a href="https://example.com">Regular link</a>
          </body>
        </html>
      `;
      const links = extractMagnetFromPage(html);

      expect(links).toHaveLength(2);
      expect(links[0]).toContain('urn:btih:abc123');
      expect(links[1]).toContain('urn:btih:def456');
    });

    it('should handle HTML with no magnet links', () => {
      const html = '<html><body><a href="https://example.com">Link</a></bo
      expect(extractMagnetFromPage(html)).toHaveLength(0);
    });
  });
});
```

Storage Manager Test (`tests/background/storage.test.ts`**)**

```
import { StorageManager } from '@/background/storage';

// Mock chrome.storage API
const mockStorage = {
  local: {
    get: jest.fn(),
    set: jest.fn(),
    remove: jest.fn(),
    clear: jest.fn()
  },
  sync: {
    get: jest.fn(),
    set: jest.fn(),
    remove: jest.fn()
  }
};

(global as any).chrome = { storage: mockStorage };

describe('StorageManager', () => {
  let storage: StorageManager;

  beforeEach(() => {
    jest.clearAllMocks();
```

```
  storage = new StorageManager();
});

describe('saveTorrent', () => {
  it('should save torrent metadata to local storage', async () => {
    const torrent = {
      infoHash: 'abc123',
      name: 'Test File',
      magnetUri: 'magnet:?xt=urn:btih:abc123',
      addedAt: Date.now()
    };

    mockStorage.local.set.mockResolvedValue(undefined);

    await storage.saveTorrent(torrent);

    expect(mockStorage.local.set).toHaveBeenCalledWith(
      expect.objectContaining({
        [`torrent_${torrent.infoHash}`]: torrent
      })
    );
  });

  it('should handle storage errors', async () => {
    mockStorage.local.set.mockRejectedValue(new Error('Storage full'));

    await expect(storage.saveTorrent({} as any)).rejects.toThrow('Storag
  });
});

describe('getTorrents', () => {
  it('should retrieve all saved torrents', async () => {
    const mockData = {
      'torrent_abc': { infoHash: 'abc', name: 'File 1' },
      'torrent_def': { infoHash: 'def', name: 'File 2' },
      'settings_key': { theme: 'dark' } // Non-torrent key
    };

    mockStorage.local.get.mockResolvedValue(mockData);

    const torrents = await storage.getTorrents();

    expect(torrents).toHaveLength(2);
    expect(torrents[0].infoHash).toBe('abc');
    expect(torrents[1].infoHash).toBe('def');
  });
});

describe('getSettings', () => {
  it('should retrieve user settings with defaults', async () => {
    mockStorage.local.get.mockResolvedValue({
      settings: { autoAddEnabled: true }
    });
```

```
      const settings = await storage.getSettings();

      expect(settings.autoAddEnabled).toBe(true);
    });

    it('should return default settings when none saved', async () => {
      mockStorage.local.get.mockResolvedValue({});

      const settings = await storage.getSettings();

      expect(settings.autoAddEnabled).toBe(false);
      expect(settings.defaultClient).toBe('transmission');
    });
  });
});
```

API Client Test (`tests/background/api-client.test.ts`**)**

```
import { ApiClient } from '@/background/api-client';
import { TorrentClientConfig } from '@/types';

// Mock fetch globally
global.fetch = jest.fn();

describe('ApiClient', () => {
  let client: ApiClient;
  const config: TorrentClientConfig = {
    type: 'transmission',
    host: 'localhost',
    port: 9091,
    username: 'admin',
    password: 'password',
    ssl: false
  };

  beforeEach(() => {
    jest.clearAllMocks();
    client = new ApiClient(config);
  });

  describe('addTorrent', () => {
    it('should successfully add torrent by magnet URI', async () => {
      const mockResponse = {
        ok: true,
        json: jest.fn().mockResolvedValue({
          arguments: { 'torrent-added': { id: 1, name: 'Test' } },
          result: 'success'
        })
      };
      (global.fetch as jest.Mock).mockResolvedValue(mockResponse);

      const result = await client.addTorrent('magnet:?xt=urn:btih:abc123')
```

```
    expect(result).toEqual({
      id: 1,
      name: 'Test',
      success: true
    });
    expect(global.fetch).toHaveBeenCalledWith(
      expect.stringContaining('/transmission/rpc'),
      expect.objectContaining({
        method: 'POST',
        body: expect.stringContaining('torrent-add')
      })
    );
  });

  it('should handle authentication failure', async () => {
    (global.fetch as jest.Mock).mockResolvedValue({
      ok: false,
      status: 401,
      statusText: 'Unauthorized'
    });

    await expect(client.addTorrent('magnet:?xt=urn:btih:abc123'))
      .rejects.toThrow('Authentication failed');
  });

  it('should handle network errors', async () => {
    (global.fetch as jest.Mock).mockRejectedValue(new Error('Network err

    await expect(client.addTorrent('magnet:?xt=urn:btih:abc123'))
      .rejects.toThrow('Network error');
  });
});

describe('getTorrents', () => {
  it('should retrieve list of active torrents', async () => {
    const mockTorrents = [
      { id: 1, name: 'File 1', status: 4, percentDone: 0.5 },
      { id: 2, name: 'File 2', status: 6, percentDone: 1.0 }
    ];
    (global.fetch as jest.Mock).mockResolvedValue({
      ok: true,
      json: jest.fn().mockResolvedValue({
        arguments: { torrents: mockTorrents },
        result: 'success'
      })
    });

    const result = await client.getTorrents();

    expect(result).toHaveLength(2);
    expect(result[0].progress).toBe(50);
    expect(result[1].isComplete).toBe(true);
  });
```

```
  });
});
```

---

4. Mocking Browser APIs

4.1 Mock Extension APIs Setup

Create `tests/setup/mock-extension-apis.ts`:

```
/**
 * Mock Chrome Extension APIs for Jest Testing
 * Based on Google's official documentation for extension unit testing
 *
 * Source: https://developer.chrome.com/docs/extensions/how-to/test/unit-t
 */

// Mock chrome global
global.chrome = {
  // Storage API
  storage: {
    local: {
      get: jest.fn((keys) => Promise.resolve({})),
      set: jest.fn((items) => Promise.resolve()),
      remove: jest.fn((keys) => Promise.resolve()),
      clear: jest.fn(() => Promise.resolve()),
    },
    sync: {
      get: jest.fn((keys) => Promise.resolve({})),
      set: jest.fn((items) => Promise.resolve()),
      remove: jest.fn((keys) => Promise.resolve()),
    },
    session: {
      get: jest.fn((keys) => Promise.resolve({})),
      set: jest.fn((items) => Promise.resolve()),
    },
    onChanged: {
      addListener: jest.fn(),
      removeListener: jest.fn(),
    },
  },

  // Tabs API
  tabs: {
    query: jest.fn((queryInfo) => Promise.resolve([])),
    get: jest.fn((tabId) => Promise.resolve({})),
    create: jest.fn((createProperties) => Promise.resolve({ id: 123 })),
    update: jest.fn((tabId, updateProperties) => Promise.resolve({})),
    remove: jest.fn((tabId) => Promise.resolve()),
    sendMessage: jest.fn((tabId, message) => Promise.resolve({})),
    onUpdated: {
      addListener: jest.fn(),
      removeListener: jest.fn(),
```

```
  },
  onActivated: {
    addListener: jest.fn(),
    removeListener: jest.fn(),
  },
},

// Runtime API
runtime: {
  sendMessage: jest.fn((message) => Promise.resolve({})),
  onMessage: {
    addListener: jest.fn(),
    removeListener: jest.fn(),
  },
  onInstalled: {
    addListener: jest.fn(),
  },
  getManifest: jest.fn(() => ({
    manifest_version: 3,
    name: 'Boba Torrent Extension',
    version: '1.0.0',
  })),
  getURL: jest.fn((path) => `chrome-extension://abc123/${path}`),
  id: 'abc123def456',
},

// Action API (Manifest V3)
action: {
  setIcon: jest.fn((details) => Promise.resolve()),
  setBadgeText: jest.fn((details) => Promise.resolve()),
  setBadgeBackgroundColor: jest.fn((details) => Promise.resolve()),
  onClicked: {
    addListener: jest.fn(),
    removeListener: jest.fn(),
  },
},

// Context menus API
contextMenus: {
  create: jest.fn((createProperties, callback) => {
    if (callback) callback();
    return 'menu-id-1';
  }),
  removeAll: jest.fn((callback) => {
    if (callback) callback();
  }),
  onClicked: {
    addListener: jest.fn(),
    removeListener: jest.fn(),
  },
},

// Notifications API
notifications: {
```

```
    create: jest.fn((notificationId, options, callback) => {
      if (callback) callback('notification-1');
      return Promise.resolve('notification-1');
    }),
    clear: jest.fn((notificationId) => Promise.resolve(true)),
    onClicked: {
      addListener: jest.fn(),
      removeListener: jest.fn(),
    },
  },

  // Web request / declarative net request
  declarativeNetRequest: {
    updateDynamicRules: jest.fn((options) => Promise.resolve()),
    getDynamicRules: jest.fn(() => Promise.resolve([])),
  },

  // Scripting API (Manifest V3)
  scripting: {
    executeScript: jest.fn((injectDetails) => Promise.resolve([])),
    insertCSS: jest.fn((details) => Promise.resolve()),
    removeCSS: jest.fn((details) => Promise.resolve()),
  },
} as any;

// Mock browser API (for Firefox polyfill compatibility)
global.browser = global.chrome;

// Mock window.matchMedia
Object.defineProperty(window, 'matchMedia', {
  writable: true,
  value: jest.fn().mockImplementation(query => ({
    matches: false,
    media: query,
    onchange: null,
    addListener: jest.fn(),
    removeListener: jest.fn(),
    addEventListener: jest.fn(),
    removeEventListener: jest.fn(),
    dispatchEvent: jest.fn(),
  })),
});

// Mock chrome.i18n
global.chrome.i18n = {
  getMessage: jest.fn((messageName, substitutions) => messageName),
  getUILanguage: jest.fn(() => 'en'),
  detectLanguage: jest.fn((text) => Promise.resolve({ isReliable: true, la
};
```

4.2 Jest Setup File

Create `tests/setup/jest-setup.ts`:

```
import '@testing-library/jest-dom';

/**
 * Custom matchers and utilities for extension testing
 */

// Matcher for checking if value is a valid magnet URI
expect.extend({
  toBeValidMagnet(received: string) {
    const pass = received.startsWith('magnet:?');
    return {
      pass,
      message: () => `expected ${received} ${pass ? 'not ' : ''}to be a va
    };
  },

  toBeHexString(received: string, length?: number) {
    const hexRegex = /^[0-9a-fA-F]+$/;
    const pass = hexRegex.test(received) && (length ? received.length ===
    return {
      pass,
      message: () => `expected ${received} ${pass ? 'not ' : ''}to be a he
    };
  }
});

// Global test utilities
global.sleep = (ms: number) => new Promise(resolve => setTimeout(resolve,

// Suppress console warnings during tests unless VERBOSE is set
if (!process.env.VERBOSE) {
  const originalWarn = console.warn;
  const originalError = console.error;

  console.warn = (...args: any[]) => {
    if (args[0]?.includes?.('React')) return;
    originalWarn.apply(console, args);
  };

  console.error = (...args: any[]) => {
    if (args[0]?.includes?.('act')) return;
    originalError.apply(console, args);
  };
}
```

4.3 Using sinon-chrome (Alternative)

```
npm install --save-dev sinon-chrome

// Alternative mocking approach using sinon-chrome
import chrome from 'sinon-chrome';

describe('Using sinon-chrome', () => {
```

```
  beforeAll(() => {
    global.chrome = chrome;
  });

  beforeEach(() => {
    chrome.flush(); // Reset all stubs between tests
  });

  test('chrome.storage.local.get', async () => {
    chrome.storage.local.get.yields({ key: 'value' });

    const result = await chrome.storage.local.get('key');
    expect(chrome.storage.local.get.calledOnce).toBe(true);
  });
});
```

---

5. E2E Testing with Playwright

5.1 Playwright Configuration

Create `playwright.config.ts`:

```
import { defineConfig, devices } from '@playwright/test';
import path from 'path';

/**
 * Playwright configuration for browser extension E2E testing
 *
 * Key requirements:
 * - Must use chromium.launchPersistentContext for extension loading
 * - Must use --disable-extensions-except and --load-extension flags
 * - Extension ID is dynamic; must extract from service worker URL
 * - workers: 1 required since persistent contexts share user data dirs
 *
 * Source: https://playwright.dev/docs/chrome-extensions
 */

export default defineConfig({
  testDir: './e2e',

  // Single worker required for persistent contexts
  workers: 1,

  // Retry in CI for flaky extension tests
  retries: process.env.CI ? 2 : 0,

  // Timeout for extension tests (slower than web tests)
  timeout: 60000,

  // Reporter configuration
  reporter: [
    ['html', { open: 'never' }],
```

```
    ['list'],
    ['junit', { outputFile: 'playwright-report/junit.xml' }]
  ],

  // Shared test settings
  use: {
    // Capture artifacts on failure
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },

  // Projects for different browsers
  projects: [
    {
      name: 'chromium',
      use: {
        ...devices['Desktop Chrome'],
        channel: 'chromium',
      },
    },
    {
      name: 'firefox',
      use: {
        ...devices['Desktop Firefox'],
      },
    },
  ],

  // Global setup/teardown
  globalSetup: require.resolve('./e2e/setup/global-setup'),
  globalTeardown: require.resolve('./e2e/setup/global-teardown'),
});
```

5.2 Playwright Test Fixtures

Create `e2e/fixtures.ts`:

```
/**
 * Playwright test fixtures for browser extension testing
 *
 * Based on official Playwright documentation for Chrome extensions:
 * - Uses chromium.launchPersistentContext for extension loading
 * - Extracts dynamic extension ID from service worker URL
 * - Provides context and extensionId to all tests
 *
 * Sources:
 * - https://playwright.dev/docs/chrome-extensions
 * - https://testdino.com/blog/browser-extensions-testing
 */

import { test as base, chromium, type BrowserContext } from '@playwright/t
import path from 'path';
```

```
// Path to built extension
const EXTENSION_PATH = path.join(__dirname, '../.output/chrome');

// Test fixture type definitions
export const test = base.extend<{
  context: BrowserContext;
  extensionId: string;
}>({
  // Create persistent context with extension loaded
  context: async ({}, use) => {
    const context = await chromium.launchPersistentContext('', {
      channel: 'chromium',
      headless: false, // Extensions require headful mode
      args: [
        `--disable-extensions-except=${EXTENSION_PATH}`,
        `--load-extension=${EXTENSION_PATH}`,
        // Additional flags for CI stability
        '--disable-dev-shm-usage',
        '--disable-gpu',
        '--no-sandbox',
      ],
      permissions: ['clipboard-read', 'clipboard-write'],
    });

    await use(context);
    await context.close();
  },

  // Extract extension ID from service worker
  extensionId: async ({ context }, use) => {
    let [serviceWorker] = context.serviceWorkers();
    if (!serviceWorker) {
      serviceWorker = await context.waitForEvent('serviceworker');
    }

    // Extension ID is the host part of chrome-extension://<id>/...
    const extensionId = serviceWorker.url().split('/')[2];
    await use(extensionId);
  },
});

export const expect = test.expect;
```

5.3 E2E Test Examples

Popup Test (`e2e/popup.spec.ts`**)**

```
import { test, expect } from './fixtures';

test.describe('Extension Popup', () => {
  test('popup renders correctly', async ({ context, extensionId }) => {
    const page = await context.newPage();
```

```
  // Navigate directly to popup page
  await page.goto(`chrome-extension://${extensionId}/popup.html`);

  // Verify popup UI elements
  await expect(page.locator('h1')).toHaveText('Boba Torrent');
  await expect(page.locator('[data-testid="add-torrent-btn"]')).toBeVisi
  await expect(page.locator('[data-testid="settings-link"]')).toBeVisibl
});

test('popup add torrent flow', async ({ context, extensionId }) => {
  const page = await context.newPage();
  await page.goto(`chrome-extension://${extensionId}/popup.html`);

  // Open add torrent dialog
  await page.click('[data-testid="add-torrent-btn"]');

  // Enter magnet URI
  const magnetInput = page.locator('[data-testid="magnet-input"]');
  await magnetInput.fill('magnet:?xt=urn:btih:1234567890abcdef1234567890

  // Submit
  await page.click('[data-testid="submit-torrent-btn"]');

  // Verify success notification
  await expect(page.locator('[data-testid="success-toast"]')).toBeVisibl
});

test('popup shows recent torrents', async ({ context, extensionId }) =>
  // Seed storage with test data via service worker
  let [sw] = context.serviceWorkers();
  if (!sw) sw = await context.waitForEvent('serviceworker');

  await sw.evaluate(() => {
    return chrome.storage.local.set({
      'torrent_abc123': {
        infoHash: 'abc123',
        name: 'Test Torrent 1',
        addedAt: Date.now(),
        status: 'downloading'
      },
      'torrent_def456': {
        infoHash: 'def456',
        name: 'Test Torrent 2',
        addedAt: Date.now() - 86400000,
        status: 'completed'
      }
    });
  });

  const page = await context.newPage();
  await page.goto(`chrome-extension://${extensionId}/popup.html`);

  // Verify torrents are displayed
```

```
    const torrents = page.locator('[data-testid="torrent-item"]');
    await expect(torrents).toHaveCount(2);
    await expect(torrents.first()).toContainText('Test Torrent 1');
  });
});
```

Content Script Test (e2e/content-script.spec.ts**)**

```
import { test, expect } from './fixtures';

test.describe('Content Script - Magnet Detection', () => {
  test('should highlight magnet links on torrent pages', async ({ context
    const page = await context.newPage();

    // Create a test page with magnet links
    await page.setContent(`
      <html>
        <body>
          <h1>Download Page</h1>
          <a href="magnet:?xt=urn:btih:abc1234567890abcdef1234567890abcdef
            Download via Magnet
          </a>
          <a href="magnet:?xt=urn:btih:def5678901234">
            Another Magnet Link
          </a>
          <a href="https://example.com/direct-download.zip">
            Direct Download
          </a>
        </body>
      </html>
    `);

    // Wait for content script to process
    await page.waitForTimeout(1000);

    // Verify magnet links are highlighted/processed
    const magnetLinks = page.locator('a[data-boba-magnet]');
    await expect(magnetLinks).toHaveCount(2);

    // Verify direct download link is NOT processed
    const directLink = page.locator('a[href="https://example.com/direct-do
    await expect(directLink).not.toHaveAttribute('data-boba-magnet');
  });

  test('should add context menu on magnet link right-click', async ({ cont
    const page = await context.newPage();

    await page.setContent(`
      <html><body>
        <a href="magnet:?xt=urn:btih:abc123" id="magnet-link">Magnet</a>
      </body></html>
    `);
```

```
    // Wait for content script
    await page.waitForTimeout(500);

    // Right-click on magnet link
    await page.click('#magnet-link', { button: 'right' });

    // In a real test, you'd verify context menu appearance
    // This requires additional setup for context menu testing
  });

  test('should NOT process non-magnet pages', async ({ context }) => {
    const page = await context.newPage();

    await page.setContent(`
      <html><body>
        <h1>Regular Article</h1>
        <p>No magnet links here</p>
        <a href="https://example.com/page">Regular link</a>
      </body></html>
    `);

    await page.waitForTimeout(500);

    // Verify no magnet processing occurred
    const processedLinks = page.locator('a[data-boba-magnet]');
    await expect(processedLinks).toHaveCount(0);
  });
});
```

Background Service Worker Test (`e2e/background.spec.ts`**)**

```
import { test, expect } from './fixtures';

test.describe('Background Service Worker', () => {
  test('service worker is active', async ({ context }) => {
    let [serviceWorker] = context.serviceWorkers();
    if (!serviceWorker) {
      serviceWorker = await context.waitForEvent('serviceworker');
    }

    // Evaluate in service worker context
    const result = await serviceWorker.evaluate(() => {
      return 'worker is alive';
    });
    expect(result).toBe('worker is alive');
  });

  test('chrome.storage interaction', async ({ context }) => {
    let [sw] = context.serviceWorkers();
    if (!sw) sw = await context.waitForEvent('serviceworker');

    // Write to storage
    await sw.evaluate(() => {
```

```
      return chrome.storage.local.set({ testKey: 'testValue' });
    });

    // Read back
    const stored = await sw.evaluate(() => {
      return chrome.storage.local.get('testKey');
    });

    expect(stored.testKey).toBe('testValue');
  });

  test('message passing between popup and background', async ({ context, e
    // Open popup
    const popupPage = await context.newPage();
    await popupPage.goto(`chrome-extension://${extensionId}/popup.html`);

    // Send message to background via popup
    const response = await popupPage.evaluate(async () => {
      return chrome.runtime.sendMessage({ action: 'ping' });
    });

    expect(response).toEqual({ status: 'ok' });
  });

  test('context menu creation', async ({ context }) => {
    let [sw] = context.serviceWorkers();
    if (!sw) sw = await context.waitForEvent('serviceworker');

    // Verify context menus were created during install
    const menus = await sw.evaluate(() => {
      // Access internal state or check chrome.contextMenus
      return chrome.contextMenus.create.toString();
    });

    expect(menus).toBeTruthy();
  });
});
```

5.4 Global Setup/Teardown

```
// e2e/setup/global-setup.ts
import { chromium } from '@playwright/test';

async function globalSetup() {
  // Ensure extension is built before tests
  // Can trigger build here if needed
  console.log('Building extension for E2E tests...');
}

export default globalSetup;

// e2e/setup/global-teardown.ts
async function globalTeardown() {
```

```
  // Cleanup any test artifacts
  console.log('E2E tests complete.');
}

export default globalTeardown;
```

---

6. Content Script Testing

6.1 DOM Scraping Logic Tests

```
// tests/content/magnet-detector.test.ts
import { MagnetDetector } from '@/content/magnet-detector';

describe('MagnetDetector', () => {
  let detector: MagnetDetector;

  beforeEach(() => {
    detector = new MagnetDetector();
    document.body.innerHTML = '';
  });

  afterEach(() => {
    document.body.innerHTML = '';
  });

  describe('findMagnetLinks', () => {
    it('should find all magnet links in DOM', () => {
      document.body.innerHTML = `
        <div>
          <a href="magnet:?xt=urn:btih:abc123">Link 1</a>
          <a href="magnet:?xt=urn:btih:def456">Link 2</a>
          <a href="https://example.com">Not magnet</a>
        </div>
      `;

      const links = detector.findMagnetLinks();

      expect(links).toHaveLength(2);
      expect(links[0].href).toContain('urn:btih:abc123');
      expect(links[1].href).toContain('urn:btih:def456');
    });

    it('should find magnet links in iframes', () => {
      document.body.innerHTML = `
        <iframe srcdoc="<a href='magnet:?xt=urn:btih:abc123'>Magnet</a>"><
      `;

      const links = detector.findMagnetLinks({ includeIframes: true });

      // Iframe traversal requires special handling
      expect(links.length).toBeGreaterThanOrEqual(0);
    });
```

```
  it('should handle pages with no magnet links', () => {
    document.body.innerHTML = `
      <div>
        <a href="https://example.com/file.zip">Download</a>
      </div>
    `;

    const links = detector.findMagnetLinks();
    expect(links).toHaveLength(0);
  });
});

describe('extractMetadata', () => {
  it('should extract metadata from magnet link element', () => {
    document.body.innerHTML = `
      <a href="magnet:?xt=urn:btih:abc123&dn=My+File"
         data-size="1.5GB"
         id="torrent-link">
        My File (1.5GB)
      </a>
    `;

    const element = document.getElementById('torrent-link') as HTMLAncho
    const metadata = detector.extractMetadata(element);

    expect(metadata.infoHash).toBe('abc123');
    expect(metadata.displayName).toBe('My File');
    expect(metadata.size).toBe('1.5GB');
  });
});

describe('processMagnetLink', () => {
  it('should add data attributes and click handler to link', () => {
    document.body.innerHTML = `
      <a href="magnet:?xt=urn:btih:abc123" id="link">Download</a>
    `;

    const link = document.getElementById('link') as HTMLAnchorElement;
    detector.processMagnetLink(link);

    expect(link.dataset.bobaMagnet).toBe('true');
    expect(link.dataset.bobaProcessed).toBe('true');
  });

  it('should not process already-processed links', () => {
    document.body.innerHTML = `
      <a href="magnet:?xt=urn:btih:abc123"
         data-boba-processed="true"
         id="link">Download</a>
    `;

    const link = document.getElementById('link') as HTMLAnchorElement;
    const addListenerSpy = jest.spyOn(link, 'addEventListener');
```

```
      detector.processMagnetLink(link);

      expect(addListenerSpy).not.toHaveBeenCalled();
    });
  });
});
```

6.2 DOM Manipulation Tests with jsdom

```
// tests/content/dom-utils.test.ts
import { DomUtils } from '@/content/dom-utils';

describe('DomUtils', () => {
  beforeEach(() => {
    document.body.innerHTML = '';
  });

  describe('createTorrentBadge', () => {
    it('should create a badge element with correct attributes', () => {
      const badge = DomUtils.createTorrentBadge({
        seeders: 150,
        leechers: 25,
        quality: '1080p'
      });

      expect(badge.tagName).toBe('SPAN');
      expect(badge.classList.contains('boba-badge')).toBe(true);
      expect(badge.textContent).toContain('150');
      expect(badge.textContent).toContain('25');
      expect(badge.textContent).toContain('1080p');
    });
  });

  describe('injectTorrentPanel', () => {
    it('should inject panel into page', () => {
      document.body.innerHTML = '<div id="container"></div>';

      const container = document.getElementById('container')!;
      const panel = DomUtils.injectTorrentPanel(container);

      expect(document.querySelector('.boba-panel')).not.toBeNull();
      expect(panel.classList.contains('boba-panel')).toBe(true);
    });

    it('should position panel correctly', () => {
      document.body.innerHTML = `
        <div id="target" style="position: relative;">
          <a href="magnet:?xt=urn:btih:abc123">Link</a>
        </div>
      `;

      const target = document.getElementById('target')!;
```

```
      const panel = DomUtils.injectTorrentPanel(target, { position: 'inlin

      expect(panel.style.position).toBe('relative');
    });
  });

  describe('observeDomChanges', () => {
    it('should call callback when DOM changes', (done) => {
      const callback = jest.fn();

      DomUtils.observeDomChanges(document.body, callback);

      // Trigger DOM mutation
      const newElement = document.createElement('div');
      document.body.appendChild(newElement);

      // Wait for MutationObserver
      setTimeout(() => {
        expect(callback).toHaveBeenCalled();
        done();
      }, 100);
    });
  });
});
```

---

7. Service Worker Testing

7.1 Background Script Tests

```
// tests/background/message-handler.test.ts
import { MessageHandler } from '@/background/message-handler';
import { ApiClient } from '@/background/api-client';

// Mock dependencies
jest.mock('@/background/api-client');

describe('MessageHandler', () => {
  let handler: MessageHandler;
  let mockApiClient: jest.Mocked<ApiClient>;
  let mockSendResponse: jest.Mock;

  beforeEach(() => {
    jest.clearAllMocks();
    mockSendResponse = jest.fn();

    // Create handler with mocked API client
    handler = new MessageHandler();
    mockApiClient = (ApiClient as jest.MockedClass<typeof ApiClient>).mock
  });

  describe('ADD_TORRENT message', () => {
    it('should add torrent via API client', async () => {
```

```
    const message = {
      action: 'ADD_TORRENT',
      magnetUri: 'magnet:?xt=urn:btih:abc123'
    };

    const mockAddTorrent = jest.fn().mockResolvedValue({
      id: 1,
      name: 'Test Torrent',
      success: true
    });

    // Override the handler's method
    handler.handleAddTorrent = mockAddTorrent;

    await handler.handleMessage(message, {}, mockSendResponse);

    expect(mockAddTorrent).toHaveBeenCalledWith(message.magnetUri);
  });

  it('should send error response on failure', async () => {
    const message = {
      action: 'ADD_TORRENT',
      magnetUri: 'invalid-magnet'
    };

    const errorResponse = { success: false, error: 'Invalid magnet URI'
    handler.handleAddTorrent = jest.fn().mockRejectedValue(new Error('In

    await handler.handleMessage(message, {}, mockSendResponse);

    // Verify error handling
    expect(mockSendResponse).toHaveBeenCalledWith(
      expect.objectContaining({ error: expect.any(String) })
    );
  });
});

describe('GET_TORRENTS message', () => {
  it('should return list of torrents', async () => {
    const message = { action: 'GET_TORRENTS' };
    const mockTorrents = [
      { id: 1, name: 'Torrent 1', progress: 50 },
      { id: 2, name: 'Torrent 2', progress: 100 }
    ];

    handler.handleGetTorrents = jest.fn().mockResolvedValue(mockTorrents

    const result = await handler.handleMessage(message, {}, mockSendResp

    expect(result).toEqual(mockTorrents);
  });
});

describe('GET_SETTINGS message', () => {
```

```
    it('should return current settings', async () => {
      const message = { action: 'GET_SETTINGS' };
      const mockSettings = {
        autoAddEnabled: false,
        defaultClient: 'transmission',
        serverUrl: 'http://localhost:9091'
      };

      handler.handleGetSettings = jest.fn().mockResolvedValue(mockSettings

      const result = await handler.handleMessage(message, {}, mockSendResp

      expect(result).toEqual(mockSettings);
    });
  });

  describe('SAVE_SETTINGS message', () => {
    it('should validate and save settings', async () => {
      const message = {
        action: 'SAVE_SETTINGS',
        settings: {
          serverUrl: 'http://localhost:9091',
          username: 'admin'
        }
      };

      handler.handleSaveSettings = jest.fn().mockResolvedValue({ success:

      const result = await handler.handleMessage(message, {}, mockSendResp

      expect(result).toEqual({ success: true });
    });
  });
});
```

7.2 API Client Tests for Torrent Clients

```
// tests/background/torrent-clients/transmission.test.ts
import { TransmissionClient } from '@/background/torrent-clients/transmiss

global.fetch = jest.fn();

describe('TransmissionClient', () => {
  let client: TransmissionClient;
  const config = {
    host: 'localhost',
    port: 9091,
    username: 'admin',
    password: 'password',
    path: '/transmission/rpc'
  };

  beforeEach(() => {
```

```
  jest.clearAllMocks();
  client = new TransmissionClient(config);
});

describe('constructor', () => {
  it('should create client with config', () => {
    expect(client).toBeDefined();
    expect(client['baseUrl']).toBe('http://localhost:9091/transmission/r
  });
});

describe('authenticate', () => {
  it('should fetch and store session ID', async () => {
    (global.fetch as jest.Mock)
      .mockRejectedValueOnce(new Response('', {
        status: 409,
        headers: { 'X-Transmission-Session-Id': 'session-id-123' }
      }))
      .mockResolvedValueOnce({
        ok: true,
        json: () => Promise.resolve({ result: 'success' })
      });

    await client.authenticate();

    expect(client['sessionId']).toBe('session-id-123');
  });
});

describe('addTorrent', () => {
  it('should send torrent-add request', async () => {
    const mockResponse = {
      ok: true,
      json: () => Promise.resolve({
        result: 'success',
        arguments: {
          'torrent-added': { id: 1, name: 'Test', hashString: 'abc123' }
        }
      })
    };
    (global.fetch as jest.Mock).mockResolvedValue(mockResponse);

    const result = await client.addTorrent('magnet:?xt=urn:btih:abc123')

    expect(global.fetch).toHaveBeenCalledWith(
      expect.any(String),
      expect.objectContaining({
        method: 'POST',
        headers: expect.objectContaining({
          'Content-Type': 'application/json'
        }),
        body: expect.stringContaining('torrent-add')
      })
    );
```

```
      expect(result.id).toBe(1);
    });
  });

  describe('getTorrents', () => {
    it('should return formatted torrent list', async () => {
      const mockResponse = {
        ok: true,
        json: () => Promise.resolve({
          result: 'success',
          arguments: {
            torrents: [
              { id: 1, name: 'File 1', status: 4, percentDone: 0.75, rateD
              { id: 2, name: 'File 2', status: 6, percentDone: 1.0, rateDo
            ]
          }
        })
      };
      (global.fetch as jest.Mock).mockResolvedValue(mockResponse);

      const torrents = await client.getTorrents();

      expect(torrents).toHaveLength(2);
      expect(torrents[0].progress).toBe(75);
      expect(torrents[1].isComplete).toBe(true);
    });
  });
});
```

---

8. Build Tool Selection

8.1 Build Tool Comparison

Tool	Dev Speed	Config Complexity	Cross-Browser	MV3 Support	Community	Verdict
WXT	Excellent (Vite-based)	Minimal	Built-in	First-class	Growing fast	**Recommended**
Extension.js	Excellent	Minimal	Built-in	First-class	Newer	Good alternative
webextension-toolbox	Good	Low	Built-in	Yes	Mature	Solid
CRXJS	Good	Low	Manual	Yes	Active	React-focused
Plasmo	Good	Minimal	Built-in	Yes	Active	Proprietary cloud
Raw Vite	Excellent	Medium	Manual	Yes	Massive	DIY approach
Raw Webpack	Slow	High	Manual	Yes	Massive	Legacy
Parcel	Good	Low	Yes	Yes	Declining	Nightly deps

8.2 Why WXT

```
Claim: WXT is built on Vite and uses Rollup for production builds, providi
Source: WXT Official Documentation / LogRocket Blog
URL: https://blog.logrocket.com/developing-web-extensions-wxt-library/
Date: 2024-06-04
Excerpt: "The WXT library uses Vite under the hood to provide features lik
Context: WXT provides file-based entrypoints, auto-imports, and automated
Confidence: High

Claim: WXT supports all browsers, both MV2 and MV3, with first-class TypeS
Source: WXT GitHub Repository
URL: https://github.com/wxt-dev/wxt
Date: 2025
Excerpt: "- World Wide Web: Supports all browsers - White Heavy Check Mark
Context: WXT is approaching v1.0 with 4K+ GitHub stars, used by extensions
Confidence: High
```

8.3 WXT Configuration

Create `wxt.config.ts`:

```
import { defineConfig } from 'wxt';
import react from '@vitejs/plugin-react';

/**
 * WXT Configuration for Boba Torrent Extension
 *
 * WXT handles:
 * - File-based entrypoints (content/, background.ts, popup.html)
 * - Cross-browser manifest generation
 * - Dev mode with HMR and auto-reload
 * - Production builds with ZIP packaging
 */

export default defineConfig({
  // Extension metadata
  manifest: {
    name: 'Boba Torrent Manager',
    description: 'Add torrents to your client directly from the browser',
    version: '1.0.0',
    permissions: [
      'storage',
      'activeTab',
      'contextMenus',
      'notifications',
      'scripting',
    ],
    host_permissions: [
      '<all_urls>'
    ],
    action: {
      default_popup: 'popup.html',
```

```
      default_icon: {
        '16': 'icon/16.png',
        '32': 'icon/32.png',
        '48': 'icon/48.png',
        '128': 'icon/128.png'
      }
    },
    icons: {
      '16': 'icon/16.png',
      '32': 'icon/32.png',
      '48': 'icon/48.png',
      '128': 'icon/128.png'
    },
  },

  // Build configuration
  vite: () => ({
    plugins: [react()],
    build: {
      sourcemap: true,
      rollupOptions: {
        output: {
          manualChunks: {
            // Split vendor code for better caching
            'vendor-ui': ['react', 'react-dom'],
          }
        }
      }
    },
    resolve: {
      alias: {
        '@': '/src',
        '@core': '/src/core',
        '@background': '/src/background',
        '@content': '/src/content',
        '@popup': '/src/popup',
        '@options': '/src/options',
      }
    }
  }),

  // Runner configuration for dev mode
  runner: {
    // Start browser with extension loaded
    startUrls: ['https://example.com']
  }
});
```

8.4 Project Structure

```
my-extension/
â”œâ”€â”€ src/
â”‚   â”œâ”€â”€ assets/          # Static assets (icons, images)
```

```
â”‚   â”œâ”€â”€ background.ts     # Service worker entrypoint
â”‚   â”œâ”€â”€ content.ts        # Content script entrypoint
â”‚   â”œâ”€â”€ content/
â”‚   â”‚   â”œâ”€â”€ magnet-detector.ts
â”‚   â”‚   â”œâ”€â”€ dom-utils.ts
â”‚   â”‚   â””â”€â”€ styles.css
â”‚   â”œâ”€â”€ popup.html        # Popup UI entrypoint
â”‚   â”œâ”€â”€ popup/
â”‚   â”‚   â”œâ”€â”€ main.tsx
â”‚   â”‚   â”œâ”€â”€ App.tsx
â”‚   â”‚   â””â”€â”€ components/
â”‚   â”œâ”€â”€ options.html      # Options page entrypoint
â”‚   â”œâ”€â”€ options/
â”‚   â”‚   â”œâ”€â”€ main.tsx
â”‚   â”‚   â””â”€â”€ App.tsx
â”‚   â”œâ”€â”€ core/             # Shared utilities
â”‚   â”‚   â”œâ”€â”€ types.ts
â”‚   â”‚   â”œâ”€â”€ constants.ts
â”‚   â”‚   â””â”€â”€ utils.ts
â”‚   â”œâ”€â”€ background/       # Background script modules
â”‚   â”‚   â”œâ”€â”€ message-handler.ts
â”‚   â”‚   â”œâ”€â”€ storage.ts
â”‚   â”‚   â”œâ”€â”€ api-client.ts
â”‚   â”‚   â””â”€â”€ torrent-clients/
â”‚   â””â”€â”€ public/           # Files copied as-is
â”‚       â””â”€â”€ _locales/
â”œâ”€â”€ tests/                # Unit tests
â”‚   â”œâ”€â”€ setup/
â”‚   â”œâ”€â”€ background/
â”‚   â”œâ”€â”€ content/
â”‚   â””â”€â”€ core/
â”œâ”€â”€ e2e/                  # Playwright E2E tests
â”‚   â”œâ”€â”€ fixtures.ts
â”‚   â”œâ”€â”€ setup/
â”‚   â”œâ”€â”€ popup.spec.ts
â”‚   â”œâ”€â”€ content-script.spec.ts
â”‚   â””â”€â”€ background.spec.ts
â”œâ”€â”€ playwright.config.ts
â”œâ”€â”€ jest.config.ts
â”œâ”€â”€ wxt.config.ts
â”œâ”€â”€ tsconfig.json
â”œâ”€â”€ eslint.config.mjs
â”œâ”€â”€ .prettierrc
â””â”€â”€ package.json
```

---

9. TypeScript Configuration

9.1 Main tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
```

```
    "module": "ESNext",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "declaration": true,
    "declarationMap": true,
    "sourceMap": true,
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"],
      "@core/*": ["src/core/*"],
      "@background/*": ["src/background/*"],
      "@content/*": ["src/content/*"],
      "@popup/*": ["src/popup/*"],
      "@options/*": ["src/options/*"]
    },
    "types": ["chrome", "node"]
  },
  "include": ["src/**/*", "tests/**/*"],
  "exclude": ["node_modules", "dist", ".output", "packages"]
}
```

9.2 Installing Type Definitions

```
# Chrome/Chromium API types
npm install --save-dev chrome-types

# Or use @types/chrome (older but widely used)
npm install --save-dev @types/chrome

# webextension-polyfill types
npm install --save-dev @types/webextension-polyfill
```

9.3 TypeScript Best Practices

```
// Use chrome-types for full API autocomplete
// Reference types at top of background files:
/// <reference types="chrome-types" />

// Define shared types in core/types.ts
export interface TorrentInfo {
  infoHash: string;
  name: string;
  magnetUri: string;
```

```
  displayName?: string;
  size?: string;
  trackers: string[];
  addedAt: number;
  status: 'pending' | 'downloading' | 'completed' | 'error';
  progress?: number;
  downloadSpeed?: number;
  uploadSpeed?: number;
  seeders?: number;
  leechers?: number;
}

export interface TorrentClientConfig {
  type: 'transmission' | 'qbittorrent' | 'deluge' | 'rtorrent';
  host: string;
  port: number;
  username?: string;
  password?: string;
  ssl?: boolean;
  path?: string;
}

export interface ExtensionSettings {
  defaultClient: string;
  autoAddEnabled: boolean;
  showNotifications: boolean;
  highlightMagnets: boolean;
  serverUrl: string;
  username: string;
  password: string;
}

// Message types for type-safe communication
export type BackgroundMessage =
  | { action: 'ADD_TORRENT'; magnetUri: string }
  | { action: 'GET_TORRENTS' }
  | { action: 'GET_SETTINGS' }
  | { action: 'SAVE_SETTINGS'; settings: Partial<ExtensionSettings> }
  | { action: 'REMOVE_TORRENT'; id: number }
  | { action: 'PING' };

export type BackgroundResponse =
  | { success: true; data?: any }
  | { success: false; error: string };
```

---

10. Linting & Code Formatting

10.1 ESLint Configuration (Flat Config for ESLint 9+)

Create `eslint.config.mjs`:

```
// @ts-check
import eslint from '@eslint/js';
import tseslint from 'typescript-eslint';
import prettier from 'eslint-plugin-prettier/recommended';

/**
 * ESLint configuration for browser extension TypeScript project
 *
 * Sources:
 * - https://oneuptime.com/blog/post/2026-02-03-eslint-prettier-typescript
 * - https://victoronsoftware.com/posts/linting-and-formatting-typescript/
 */

export default tseslint.config(
  // Base recommended JS rules
  eslint.configs.recommended,

  // TypeScript recommended + type-checked rules
  ...tseslint.configs.recommendedTypeChecked,
  ...tseslint.configs.stylisticTypeChecked,

  // Global ignores
  {
    ignores: [
      'dist/**/*',
      '.output/**/*',
      'packages/**/*',
      'coverage/**/*',
      'node_modules/**/*',
      'eslint.config.mjs',
      'playwright.config.ts',
      'jest.config.ts',
      'wxt.config.ts'
    ]
  },

  // Language options for type checking
  {
    languageOptions: {
      parserOptions: {
        project: './tsconfig.json',
        tsconfigRootDir: import.meta.dirname,
      },
      globals: {
        // Browser globals
        chrome: 'readonly',
        browser: 'readonly',
        window: 'readonly',
        document: 'readonly',
        // Node globals for config files
        process: 'readonly',
        __dirname: 'readonly'
      }
    }
```

```
  },

  // Custom rules
  {
    rules: {
      // Code quality
      'no-console': ['warn', { allow: ['warn', 'error'] }],
      'no-debugger': 'warn',

      // TypeScript strictness
      '@typescript-eslint/explicit-function-return-type': 'off',
      '@typescript-eslint/no-explicit-any': 'warn',
      '@typescript-eslint/no-unused-vars': [
        'error',
        { argsIgnorePattern: '^_', varsIgnorePattern: '^_' }
      ],
      '@typescript-eslint/no-floating-promises': 'error',
      '@typescript-eslint/await-thenable': 'error',
      '@typescript-eslint/no-misused-promises': 'error',

      // Import/exports
      '@typescript-eslint/consistent-type-imports': [
        'error',
        { prefer: 'type-imports', fixStyle: 'separate-type-imports' }
      ],

      // Style
      '@typescript-eslint/prefer-nullish-coalescing': 'warn',
      '@typescript-eslint/prefer-optional-chain': 'warn',
    }
  },

  // Prettier must be last
  prettier
);
```

10.2 Prettier Configuration

Create `.prettierrc`:

```
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5",
  "printWidth": 100,
  "bracketSpacing": true,
  "arrowParens": "avoid",
  "endOfLine": "lf"
}
```

Create `.prettierignore`:

```
dist/
.output/
coverage/
packages/
node_modules/
*.min.js
package-lock.json
CHANGELOG.md
```

10.3 VS Code Integration

Create `.vscode/settings.json`:

```
{
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  },
  "eslint.validate": [
    "javascript",
    "typescript"
  ],
  "typescript.preferences.importModuleSpecifier": "relative"
}
```

Create `.vscode/extensions.json`:

```
{
  "recommendations": [
    "dbaeumer.vscode-eslint",
    "esbenp.prettier-vscode",
    "bradlc.vscode-tailwindcss",
    "ms-playwright.playwright"
  ]
}
```

---

11. CI/CD Pipeline

11.1 GitHub Actions â€” Main CI Workflow

Create `.github/workflows/ci.yml`:

```
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

concurrency:
```

```
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  # â”€â”€â”€ Lint & Type Check â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
  lint:
    name: Lint & Type Check
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run ESLint
        run: npm run lint

      - name: Check formatting
        run: npm run format:check

      - name: Type check
        run: npm run typecheck

  # â”€â”€â”€ Unit Tests â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â
  unit-tests:
    name: Unit Tests
    runs-on: ubuntu-latest
    needs: lint
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run unit tests
        run: npm run test:ci

      - name: Upload coverage report
        uses: actions/upload-artifact@v4
        if: always()
```

```
      with:
        name: coverage-report
        path: coverage/
        retention-days: 7

    - name: Publish test results
      uses: dorny/test-reporter@v1
      if: always()
      with:
        name: Jest Tests
        path: junit.xml
        reporter: jest-junit

# â”€â”€â”€ Build â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
build:
  name: Build (${{ matrix.browser }})
  runs-on: ubuntu-latest
  needs: lint
  strategy:
    matrix:
      browser: [chrome, firefox, edge]
    fail-fast: false
  steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: 20
        cache: 'npm'

    - name: Install dependencies
      run: npm ci

    - name: Build for ${{ matrix.browser }}
      run: npx wxt build -b ${{ matrix.browser }}

    - name: Create ZIP package
      run: npx wxt zip -b ${{ matrix.browser }}

    - name: Upload build artifact
      uses: actions/upload-artifact@v4
      with:
        name: extension-${{ matrix.browser }}
        path: .output/${{ matrix.browser }}/*.zip
        retention-days: 14

# â”€â”€â”€ E2E Tests â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”
e2e-tests:
  name: E2E Tests (Chromium)
  runs-on: ubuntu-latest
  needs: build
  steps:
```

```yaml
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Install Playwright browsers
        run: npx playwright install --with-deps chromium

      - name: Build extension for testing
        run: npx wxt build -b chrome

      - name: Run Playwright tests
        run: npx playwright test

      - name: Upload Playwright report
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: |
            playwright-report/
            test-results/
          retention-days: 14

  # â”€â”€â”€ Manifest Validation â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€€â
  manifest-check:
    name: Validate Manifest
    runs-on: ubuntu-latest
    needs: build
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install web-ext
        run: npm install -g web-ext

      - name: Validate Firefox manifest
        run: web-ext lint --source-dir .output/firefox/

      - name: Validate Chrome manifest (schema check)
        run: |
```

```
          # Check manifest is valid JSON
          cat .output/chrome/manifest.json | python3 -m json.tool > /dev/n
          echo "Chrome manifest is valid JSON"
```

11.2 GitHub Actions â€” Release Workflow

Create `.github/workflows/release.yml`:

```
name: Release

on:
  push:
    branches: [main]

permissions:
  contents: write
  pull-requests: write

jobs:
  # â”€â”€â”€ Release Please â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
  release-please:
    name: Create Release
    runs-on: ubuntu-latest
    outputs:
      release_created: ${{ steps.release.outputs.release_created }}
      tag_name: ${{ steps.release.outputs.tag_name }}
    steps:
      - name: Run Release Please
        uses: googleapis/release-please-action@v4
        id: release
        with:
          token: ${{ secrets.GITHUB_TOKEN }}
          release-type: node
          config-file: release-please-config.json
          manifest-file: .release-please-manifest.json

  # â”€â”€â”€ Publish Chrome Web Store â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
  publish-chrome:
    name: Publish to Chrome Web Store
    needs: release-please
    if: ${{ needs.release-please.outputs.release_created }}
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
```

```
        run: npm ci

      - name: Build and package for Chrome
        run: |
          npx wxt build -b chrome
          npx wxt zip -b chrome

      - name: Upload to Chrome Web Store
        uses: mobilefirstllc/cws-publish@latest
        with:
          action: 'publish'
          client_id: ${{ secrets.CHROME_CLIENT_ID }}
          client_secret: ${{ secrets.CHROME_CLIENT_SECRET }}
          refresh_token: ${{ secrets.CHROME_REFRESH_TOKEN }}
          extension_id: ${{ secrets.CHROME_EXTENSION_ID }}
          zip_file: .output/chrome/boba-torrent-manager-chrome.zip

  # â”€â”€â”€ Publish Firefox AMO â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
  publish-firefox:
    name: Publish to Firefox AMO
    needs: release-please
    if: ${{ needs.release-please.outputs.release_created }}
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Build and package for Firefox
        run: |
          npx wxt build -b firefox
          npx wxt zip -b firefox

      - name: Sign and publish to AMO
        uses: kewisch/action-web-ext@v1
        with:
          cmd: sign
          source: .output/firefox/boba-torrent-manager-firefox.zip
          channel: listed
          apiKey: ${{ secrets.AMO_API_KEY }}
          apiSecret: ${{ secrets.AMO_API_SECRET }}
          timeout: 900000

  # â”€â”€â”€ Publish Edge Add-ons â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
  publish-edge:
    name: Publish to Edge Add-ons
```

```
    needs: release-please
    if: ${{ needs.release-please.outputs.release_created }}
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Build and package for Edge
        run: |
          npx wxt build -b edge
          npx wxt zip -b edge

      # Edge requires manual upload to Partner Center
      # Upload the ZIP as a workflow artifact for manual submission
      - name: Upload Edge package
        uses: actions/upload-artifact@v4
        with:
          name: edge-package
          path: .output/edge/*.zip
          retention-days: 30

  # â”€â”€â”€ Publish GitHub Release Assets â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
  release-assets:
    name: Upload Release Assets
    needs: [release-please, publish-chrome, publish-firefox]
    if: ${{ needs.release-please.outputs.release_created }}
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Build all packages
        run: |
          npm ci
          npx wxt zip -b chrome
          npx wxt zip -b firefox
          npx wxt zip -b edge

      - name: Upload release assets
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
```

```
          gh release upload ${{ needs.release-please.outputs.tag_name }} \
            .output/chrome/*.zip \
            .output/firefox/*.zip \
            .output/edge/*.zip
```

11.3 Release Please Configuration

Create `release-please-config.json`:

```
{
  "packages": {
    ".": {
      "release-type": "node",
      "changelog-path": "CHANGELOG.md",
      "bump-minor-pre-major": true,
      "bump-patch-for-minor-pre-major": true,
      "draft": false,
      "prerelease": false,
      "extra-files": [
        "wxt.config.ts",
        "src/public/manifest.json"
      ]
    }
  }
}
```

Create `.release-please-manifest.json`:

```
{
  ".": "1.0.0"
}
```

---

12. Extension Packaging

12.1 Build and ZIP with WXT

```
# Development build for Chrome
npm run dev

# Production build for specific browser
npx wxt build -b chrome
npx wxt build -b firefox
npx wxt build -b edge
npx wxt build -b opera
npx wxt build -b safari

# Create ZIP packages for store submission
npx wxt zip -b chrome     # â†’ .output/chrome/extension.zip
npx wxt zip -b firefox    # â†’ .output/firefox/extension.xpi
npx wxt zip -b edge       # â†’ .output/edge/extension.zip
```

```
# Build all browsers at once
npm run build:all
```

12.2 Manual ZIP Creation (Script)

Create `scripts/package.sh`:

```
#!/bin/bash
set -euo pipefail

# Cross-browser extension packaging script

BROWSERS=("chrome" "firefox" "edge")
VERSION=$(node -p "require('./package.json').version")
OUTPUT_DIR="packages"

mkdir -p "$OUTPUT_DIR"

for browser in "${BROWSERS[@]}"; do
  echo "Packaging for $browser..."

  # Build for browser
  npx wxt build -b "$browser"

  # Create package
  npx wxt zip -b "$browser"

  # Rename with version
  src=".output/$browser/boba-torrent-manager-$browser.zip"
  dst="$OUTPUT_DIR/boba-torrent-manager-v${VERSION}-${browser}.zip"

  if [ -f "$src" ]; then
    cp "$src" "$dst"
    echo "Created: $dst"
  fi
done

echo "Packaging complete! Files in $OUTPUT_DIR:"
ls -la "$OUTPUT_DIR/"
```

12.3 Manifest Validation

```
# Install Mozilla's web-ext for linting
npm install -g web-ext

# Validate manifest for Firefox
web-ext lint --source-dir .output/firefox/ --output json

# Using web-ext in CI
web-ext lint --source-dir .output/firefox/ --warnings-as-errors
```

12.4 CRX Generation (for self-distribution)

```
# Install CRX tool
npm install -g crx

# Generate CRX with private key
crx pack .output/chrome -o packages/extension.crx \
  -p .keys/chrome-private-key.pem

# Generate update.xml for auto-updates
echo '<?xml version="1.0" encoding="UTF-8"?>
<gupdate xmlns="http://www.google.com/update2/response" protocol="2.0">
  <app appid="YOUR_EXTENSION_ID">
    <updatecheck codebase="https://yourserver.com/extension.crx" version="
  </app>
</gupdate>' > packages/update.xml
```

---

13. Store Submission & Distribution

13.1 Store Requirements Summary

Store	Registration Fee	Upload Method	Review Time	Auto-Publish	2FA Required
Chrome Web Store	$5 one-time	API (CI)	< 1 hour (auto)	Yes	Yes
Firefox AMO	Free	web-ext CLI	~seconds auto + manual after	Partial	Yes
Edge Add-ons	Free	Manual (Partner Center)	Up to 7 days	No	Yes
Opera Add-ons	Free	Manual upload	Manual review	No	No
Yandex	Free	Manual upload	Unknown	No	No

13.2 Chrome Web Store

Setup: 1. Pay $5 developer registration fee 2. Create OAuth 2.0 credentials in Google Cloud Console 3. Enable Chrome Web Store API 4. Generate refresh token using OAuth flow

Required Secrets:

```
CHROME_CLIENT_ID     - OAuth 2.0 Client ID
CHROME_CLIENT_SECRET - OAuth 2.0 Client Secret
CHROME_REFRESH_TOKEN - OAuth 2.0 Refresh Token
CHROME_EXTENSION_ID  - Extension ID from CWS
```

Publishing via API:

```
# Using chrome-webstore-upload CLI
npm install -g chrome-webstore-upload-cli

chrome-webstore-upload upload \
  --source extension.zip \
  --extension-id $EXTENSION_ID \
  --client-id $CLIENT_ID \
  --client-secret $CLIENT_SECRET \
  --refresh-token $REFRESH_TOKEN
```

Chrome Web Store API Setup Steps:

```
1. Go to https://console.cloud.google.com/
2. Create new project
3. Enable "Chrome Web Store API" in API Library
4. Go to Credentials â†’ Create Credentials â†’ OAuth client ID
5. Choose "Desktop App" as application type
6. Download client credentials
7. Run OAuth flow to get refresh token:
   https://github.com/fregante/chrome-webstore-upload/blob/main/How%20to%2
```

13.3 Firefox AMO

Setup: 1. Create Firefox Account 2. Go to https://addons.mozilla.org/developers/addon/api/key/
3. Generate API credentials (JWT issuer + JWT secret)

Publishing via web-ext:

```
# Get API credentials from AMO
export AMO_JWT_ISSUER=your-jwt-issuer
export AMO_JWT_SECRET=your-jwt-secret

# Build and sign
npx web-ext sign \
  --source-dir .output/firefox/ \
  --channel=listed \
  --api-key=$AMO_JWT_ISSUER \
  --api-secret=$AMO_JWT_SECRET \
  --timeout=900000
```

Firefox Distribution:

```
Claim: web-ext supports lint, build, and sign commands for Firefox extensi
Source: extensionworkshop.com
URL: https://extensionworkshop.com/documentation/develop/getting-started-w
Date: 2026-03-22
Excerpt: "Start using the web-ext command-line tool. Automate and simplify
Context: web-ext is Mozilla's official CLI tool
Confidence: High
```

13.4 Edge Add-ons

Setup: 1. Register at Microsoft Partner Center (free) 2. Enroll in Microsoft Edge program 3.
Submit extension manually via Partner Center dashboard

Note: Edge requires manual submission for the first version. Subsequent updates can potentially be automated but require additional setup.

13.5 Opera Add-ons

Submission: 1. Go to https://addons.opera.com/developer/ 2. Sign in with Opera account 3. Upload extension via â€œUpload Extensionâ€ form 4. Fill in extension details, screenshots, description 5. Submit for manual review

```
Claim: Opera extensions are reviewed manually with no SLA provided.
Source: Opera Help - Publishing Guidelines
URL: https://help.opera.com/en/extensions/publishing-guidelines/
Date: 2026-03-09
Excerpt: "When you submit your extension, we will evaluate it according to
Context: Opera accepts standard Chromium extensions (.zip format)
Confidence: High
```

13.6 Store Submission Checklist

```
## Pre-Submission Checklist

### All Stores
- [ ] Manifest version is updated
- [ ] Icons present at all sizes (16, 32, 48, 128, 512)
- [ ] Description is clear and concise
- [ ] Screenshots provided (640x480 or 1280x800)
- [ ] Privacy policy URL included
- [ ] No remote code execution
- [ ] Content Security Policy set
- [ ] No console.log statements in production
- [ ] Source maps removed (or included per policy)

### Chrome Web Store
- [ ] $5 developer fee paid
- [ ] OAuth credentials configured
- [ ] Extension ID is consistent
- [ ] Store listing description optimized
- [ ] Category selected appropriately
- [ ] Promotional images (optional)
- [ ] Privacy practices declared

### Firefox AMO
- [ ] API key generated
- [ ] browser_specific_settings.gecko.id set in manifest
- [ ] Source code submitted (if minified/obfuscated)
- [ ] Permissions justified

### Edge Add-ons
- [ ] Partner Center account verified
- [ ] Maturity content flag checked
- [ ] Certification notes prepared
- [ ] Support contact info provided
```

---

14. Version Management

14.1 Semantic Versioning

```
Claim: Conventional Commits (fix:, feat:, BREAKING CHANGE:) map to SemVer
Source: Conventional Commits specification
URL: https://www.conventionalcommits.org/en/v1.0.0/
Date: N/A
Excerpt: "fix: a commit that patches a bug â†’ PATCH; feat: a commit that
Context: Industry standard for automated versioning
Confidence: High
```

14.2 Commit Convention

```
## Commit Message Format

<type>(<scope>): <subject>

[body]

[footer]

## Types
- `feat`: New feature (minor version bump)
- `fix`: Bug fix (patch version bump)
- `docs`: Documentation changes
- `style`: Code style changes (formatting, etc.)
- `refactor`: Code refactoring
- `perf`: Performance improvements
- `test`: Adding or updating tests
- `chore`: Build process or auxiliary tool changes
- `ci`: CI/CD changes

## Scopes
- `popup`: Popup UI
- `content`: Content script
- `bg`: Background script
- `api`: API client
- `storage`: Storage manager
- `build`: Build system
- `docs`: Documentation

## Examples
feat(popup): add torrent list with search filter
fix(bg): handle storage quota exceeded error
docs: update installation instructions
test(api): add tests for qBittorrent client
```

14.3 Version Syncing

The release-please configuration ensures version sync across files:

```
// release-please-config.json
{
  "packages": {
    ".": {
      "release-type": "node",
      "extra-files": [
        {
          "type": "json",
          "path": "src/public/manifest.json",
          "jsonpath": "$.version"
        },
        {
          "type": "ts",
          "path": "wxt.config.ts",
          "jsonpath": "$.manifest.version"
        }
      ]
    }
  }
}
```

---

15. Code Quality & Coverage

15.1 Jest Coverage Configuration

Jest has built-in coverage support via Istanbul:

```
// jest.config.ts
{
  "collectCoverageFrom": [
    "src/**/*.{ts,tsx}",
    "!src/**/*.d.ts",
    "!src/**/index.ts",
    "!src/entrypoints/**"
  ],
  "coverageThreshold": {
    "global": {
      "branches": 70,
      "functions": 70,
      "lines": 70,
      "statements": 70
    }
  },
  "coverageReporters": ["text", "text-summary", "lcov", "html", "json-summ
}
```

15.2 Coverage in CI

```
# In GitHub Actions workflow
- name: Run tests with coverage
  run: npm run test:coverage
```

```
- name: Upload coverage to Codecov
  uses: codecov/codecov-action@v4
  with:
    files: ./coverage/lcov.info
    fail_ci_if_error: true

- name: Upload coverage to Coveralls
  uses: coverallsapp/github-action@v2
  with:
    github-token: ${{ secrets.GITHUB_TOKEN }}
```

15.3 SonarQube Integration (Optional)

```
# .github/workflows/sonar.yml
name: SonarQube Analysis
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  sonar:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run SonarQube Scan
        uses: SonarSource/sonarqube-scan-action@v4
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
```

Create `sonar-project.properties`:

```
sonar.projectKey=boba-extension
sonar.organization=boba
sonar.sources=src
sonar.tests=tests,e2e
sonar.typescript.lcov.reportPaths=coverage/lcov.info
sonar.coverage.exclusions=**/*.test.ts,**/entrypoints/**
sonar.exclusions=node_modules/**,dist/**,.output/**
```

15.4 Coverage Badges

```
<!-- README.md -->
![Coverage](https://img.shields.io/codecov/c/github/your-org/boba-extensio
![Tests](https://img.shields.io/github/actions/workflow/status/your-org/bo
![Version](https://img.shields.io/github/v/release/your-org/boba-extension
```

---

16. Cross-Browser Build System

16.1 WXT Cross-Browser Builds

WXT handles cross-browser builds automatically:

```
# Build for all supported browsers
npx wxt build -b chrome    # Manifest V3, service worker
npx wxt build -b firefox   # Manifest V2/V3, background script
npx wxt build -b edge      # Manifest V3, same as Chrome
npx wxt build -b opera     # Manifest V3, Chromium-based
npx wxt build -b safari    # Manifest V2 (if supported)
```

16.2 Browser-Specific Code

```
// src/core/browser-detector.ts
export function getBrowser(): 'chrome' | 'firefox' | 'edge' | 'opera' | 's
  if (typeof chrome !== 'undefined') {
    if (chrome.runtime.getManifest().browser_specific_settings?.gecko) {
      return 'firefox';
    }
    if (navigator.userAgent.includes('Edg/')) return 'edge';
    if (navigator.userAgent.includes('OPR/')) return 'opera';
    if (navigator.userAgent.includes('Safari/') && !navigator.userAgent.in
      return 'safari';
    }
    return 'chrome';
  }
  return 'unknown';
}

export function isManifestV3(): boolean {
  return chrome.runtime.getManifest().manifest_version === 3;
}

// Use browser-specific APIs
export async function getBackgroundPage(): Promise<Window | null> {
  const browser = getBrowser();
  if (browser === 'firefox') {
    // Firefox supports background pages in MV2
    return chrome.extension.getBackgroundPage();
  }
  // Chrome MV3 uses service workers
  return null;
}
```

16.3 WebExtension Polyfill

```
npm install webextension-polyfill
npm install --save-dev @types/webextension-polyfill

// Use browser.* API with Promise support everywhere
import browser from 'webextension-polyfill';
```

```
// Works in both Chrome and Firefox with Promises
const tabs = await browser.tabs.query({ active: true, currentWindow: true
await browser.storage.local.set({ key: 'value' });
```

16.4 Build Matrix in CI

```
strategy:
  matrix:
    browser: [chrome, firefox, edge]
    node-version: [18, 20, 22]
    exclude:
      # Edge builds only on Node 20+
      - browser: edge
        node-version: 18
  fail-fast: false
```

---

17. Pre-Commit Hooks

17.1 Husky + lint-staged Setup

```
# Install
npm install --save-dev husky lint-staged

# Initialize
npx husky init
```

17.2 Pre-Commit Hook

Create `.husky/pre-commit`:

```
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"

npx lint-staged
```

17.3 lint-staged Configuration

Add to `package.json`:

```
{
  "lint-staged": {
    "*.{ts,tsx}": [
      "eslint --fix",
      "prettier --write",
      "jest --findRelatedTests --bail --passWithNoTests"
    ],
    "*.{js,jsx}": [
      "eslint --fix",
      "prettier --write"
    ],
```

```
    "*.{json,md,yml,yaml}": [
      "prettier --write"
    ],
    "*.css": [
      "prettier --write"
    ]
  }
}
```

17.4 Commit Message Hook (Optional)

Create `.husky/commit-msg`:

```
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"

# Validate conventional commit format
npx commitlint --edit $1
```

Install commitlint:

```
npm install --save-dev @commitlint/config-conventional @commitlint/cli
```

Create `.commitlintrc.json`:

```
{
  "extends": ["@commitlint/config-conventional"],
  "rules": {
    "type-enum": [
      2,
      "always",
      ["feat", "fix", "docs", "style", "refactor", "perf", "test", "chore"
    ],
    "scope-enum": [
      2,
      "always",
      ["popup", "content", "bg", "api", "storage", "build", "deps", "docs"
    ]
  }
}
```

---

18. Development Workflow README

Create `README.md`:

```
# Boba Torrent Extension

Cross-browser torrent management extension with support for Chrome, Firefo

## Quick Start

```bash
```
```

```
# Install dependencies
npm install

# Start development server (Chrome)
npm run dev

# Start development server (Firefox)
npm run dev:firefox
```

Development Scripts

Command	Description
`npm run dev`	Start dev server (Chrome)
`npm run dev:firefox`	Start dev server (Firefox)
`npm run build`	Build for Chrome
`npm run build:firefox`	Build for Firefox
`npm run build:all`	Build for all browsers
`npm run zip`	Create ZIP for Chrome
`npm run zip:all`	Create ZIPs for all browsers
`npm test`	Run unit tests
`npm run test:watch`	Run tests in watch mode
`npm run test:coverage`	Run tests with coverage
`npm run test:e2e`	Run Playwright E2E tests
`npm run lint`	Run ESLint
`npm run lint:fix`	Fix ESLint issues
`npm run format`	Format with Prettier
`npm run typecheck`	TypeScript type check

Project Structure

```
src/
  background.ts       # Service worker / background script
  content.ts          # Content script entry point
  popup.html          # Popup UI
  options.html        # Options page
  core/               # Shared utilities and types
  background/         # Background script modules
  content/            # Content script modules
  popup/              # Popup UI components
  options/            # Options page components
  assets/             # Static assets (icons, etc.)
tests/                # Unit tests
e2e/                  # Playwright E2E tests
```

Testing

Unit Tests

Unit tests use Jest with jsdom environment. Run with:

```
npm test              # Single run
npm run test:watch    # Watch mode
npm run test:coverage # With coverage report
```

Tests are organized by module: - `tests/background/` - Background script tests - `tests/content/` - Content script tests - `tests/core/` - Utility and type tests

E2E Tests

E2E tests use Playwright. Run with:

```
npm run test:e2e      # Headless mode
npm run test:e2e:ui   # Interactive UI mode
```

Note: E2E tests require the extension to be built first.

Writing Tests

See the test examples in: - `tests/background/magnet-parser.test.ts` - `tests/background/storage.test.ts` - `e2e/popup.spec.ts` - `e2e/content-script.spec.ts`

Code Style

This project uses: - **ESLint** for code quality - **Prettier** for formatting - **TypeScript** strict mode

Pre-commit hooks automatically lint and format staged files.

Building for Stores

```
# Create packages for all stores
npm run build:all
npm run zip:all

# Packages will be in .output/<browser>/
```

Release Process

Releases are automated via Release Please:

1. Merge PRs with Conventional Commits
2. Release Please creates a release PR automatically
3. Merge the release PR
4. GitHub Release is created automatically
5. Extension is published to stores automatically

Manual Version Bump

```
# Patch version
npm version patch

# Minor version
npm version minor

# Major version
npm version major
```

Browser Support

Browser	Min Version	Manifest
Chrome	88+	V3
Firefox	109+	V2/V3
Edge	88+	V3
Opera	74+	V3

Contributing

1. Fork the repository
2. Create a feature branch (`git checkout -b feat/amazing-feature`)
3. Make changes with tests
4. Commit using conventional commits (`feat: add amazing feature`)
5. Push and create a Pull Request

License

MIT

---

19. Source Citations

Testing Frameworks

Claim: Jest with setupFiles can mock chrome.* APIs globally for extension unit tests. Source: Chrome Developers - Unit testing Chrome Extensions URL: https://developer.chrome.com/docs/extensions/how-to/test/unit-testing Date: 2023-10-12 Excerpt: â€œCreate a jest.config.js file that declares a setup fileâ€¦ In mock-extension-apis.js, add implementations for the specific functions you call.â€ Context: Official Google documentation for extension testing Confidence: High

Claim: Playwright supports Chrome extension testing via chromium.launchPersistentContext with â€“load-extension and â€“disable-extensions-except flags. Source: Playwright Official Documentation - Chrome Extensions URL: https://playwright.dev/docs/chrome-extensions Date: 2025 Excerpt: â€œThe recommended approach is to use a fixtureâ€¦ chromium.launchPersistentContext creates a full user profile directory and loads your unpacked

extension via Chrome command-line flags.â€ Context: Official Playwright documentation Confidence: High

Claim: Extension testing requires persistent browser contexts since extensions attach to browser profiles at launch time, not to individual tabs. Source: TestDino - Testing Browser Extensions with Playwright URL: https://testdino.com/blog/browser-extensions-testing Date: 2026-05-04 Excerpt: â€œExtension testing requires a persistent browser context. This is because extensions attach to the browser profile at launch time, not to individual tabs.â€ Context: Tutorial on extension testing with Playwright Confidence: High

Build Tools

Claim: WXT uses Vite under the hood with Rollup for production, providing HMR and fast reloads. Source: LogRocket - Developing web extensions with the WXT library URL: https://blog.logrocket.com/developing-web-extensions-wxt-library/ Date: 2024-06-04 Excerpt: â€œThe WXT library uses Vite under the hood to provide features like HMRâ€¦ WXT allows you to use any framework of your choice.â€ Context: Technical tutorial on WXT Confidence: High

Claim: Vite is significantly faster than Webpack for development: <1s cold start vs 5-10s, <50ms HMR vs 1.5-3s. Source: Tech Insider - Vite vs Webpack URL: https://tech-insider.org/vite-vs-webpack-2026/ Date: 2026-05-30 Excerpt: â€œDev Cold Start (large app, ~1,000 modules): <1s vs 5-10s. HMR Update (single file change): <50ms vs 1.5-3s.â€ Context: Comprehensive benchmark comparison Confidence: High

Claim: Extension.js framework provides browser-prefixed manifest keys (chrome:, firefox:, edge:) that filter at compile time. Source: Extension.js Official Website URL: https://extension.js.org/ Date: 2026-05-29 Excerpt: â€œBrowser-prefixed keys (chrome:, firefox:, edge:) filter at compile time. Unprefixed keys apply everywhere.â€ Context: Official framework documentation Confidence: High

CI/CD & Store Distribution

Claim: Chrome Web Store supports automated publishing via API with OAuth 2.0 credentials. Source: Chrome Web Store Upload Action - GitHub Marketplace URL: https://github.com/marketplace/actions/publish-chrome-extension-to-chrome-web-store Date: N/A Excerpt: â€œUpload chrome extensions to Chrome Web Store programmatically using CI/CD pipeline.â€ Context: GitHub Action for automated publishing Confidence: High

Claim: release-please creates Release PRs based on Conventional Commits, allowing verification before automated publishing. Source: Zenn.dev - Chrome Extension Auto-Publish Guide URL: https://zenn.dev/atani/articles/chrome-extension-auto-publish-guide Date: 2026-01-20 Excerpt: â€œrelease-please uses a Release PR, allowing you to verify the CHANGELOG and version number before releasing.â€ Context: Step-by-step CI/CD setup guide Confidence: High

Claim: Firefox extensions must be signed by Mozilla through AMO before installation in release versions. Source: Extension Workshop - Signing and distribution overview URL: https://extensionworkshop.com/documentation/publish/signing-and-distribution-overview/ Date: 2026-03-22 Excerpt: â€œAdd-ons need to be signed before they can be installed into release and beta versions of Firefox.â€ Context: Official Mozilla documentation Confidence: High

Claim: web-ext CLI supports lint, build, sign, and run commands for Firefox extensions. Source: Mozilla/web-ext GitHub URL: https://github.com/mozilla/web-ext Date: 2025 Excerpt: â€œA command line tool to help build, run, and test WebExtensions.â€ Context: Official Mozilla CLI tool Confidence: High

Code Quality

Claim: ESLint 9+ uses flat config (eslint.config.mjs) which replaces the legacy .eslintrc format. Source: OneUptime - Configure ESLint and Prettier for TypeScript URL: https://oneuptime.com/blog/post/2026-02-03-eslint-prettier-typescript/view Date: 2026-02-03 Excerpt: â€œIf you are using ESLint 9 or planning to upgrade, the configuration system has changed to flat config using eslint.config.js.â€ Context: Modern ESLint setup guide Confidence: High

Claim: Husky 9+ uses .husky/ directory with core.hooksPath. lint-staged runs tasks only on staged files. Source: Steve Kinney - Husky and lint-staged URL: https://stevekinney.com/courses/enterprise-ui/husky-and-lint-staged Date: 2026-03-17 Excerpt: â€œHusky is the thin layer that manages those native hooksâ€¦ lint-staged is the thing you usually run inside pre-commit to execute commands only on staged files.â€ Context: Detailed explanation of modern Husky setup Confidence: High

Cross-Browser Development

Claim: MDN recommends using webextension-polyfill for cross-browser compatibility, coding for Firefox with Promises and using the polyfill for Chrome. Source: MDN - Build a cross-browser extension URL: https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Build_a_cross_browser_extension Date: 2025-07-17 Excerpt: â€œThe solution is to code for Firefox using promises and use the WebExtension browser API Polyfill to address Chrome, Opera, and Edge.â€ Context: Official MDN documentation Confidence: High

Claim: Chrome requires service_worker in MV3 while Firefox requires scripts for background entries, requiring different manifest configurations per browser. Source: Stack Overflow - Chrome and Firefox MV3 compatibility URL: https://stackoverflow.com/questions/78491335 Date: 2026-02-22 Excerpt: â€œChrome requires the use of service_worker while Firefox still requires using scripts.â€ Context: Real-world cross-browser compatibility issue Confidence: High

---

Appendix A: Package.json Complete Example

```json
{
  "name": "boba-torrent-extension",
  "version": "1.0.0",
  "description": "Cross-browser torrent manager extension",
  "type": "module",
  "scripts": {
    "dev": "wxt",
    "dev:firefox": "wxt -b firefox",
```

```
    "build": "wxt build",
    "build:firefox": "wxt build -b firefox",
    "build:edge": "wxt build -b edge",
    "build:all": "wxt build -b chrome && wxt build -b firefox && wxt build
    "zip": "wxt zip",
    "zip:all": "wxt zip -b chrome && wxt zip -b firefox && wxt zip -b edge
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage",
    "test:ci": "jest --ci --coverage --reporters=default --reporters=jest-
    "test:e2e": "playwright test",
    "test:e2e:ui": "playwright test --ui",
    "lint": "eslint src tests e2e --ext .ts,.tsx",
    "lint:fix": "eslint src tests e2e --ext .ts,.tsx --fix",
    "format": "prettier --write \"src/**/*.{ts,tsx,json,css}\" \"tests/**/
    "format:check": "prettier --check \"src/**/*.{ts,tsx,json,css}\" \"tes
    "typecheck": "tsc --noEmit",
    "prepare": "husky",
    "postinstall": "wxt prepare"
  },
  "dependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "webextension-polyfill": "^0.12.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.17.0",
    "@playwright/test": "^1.49.0",
    "@testing-library/jest-dom": "^6.6.0",
    "@types/chrome": "^0.0.287",
    "@types/jest": "^29.5.0",
    "@types/react": "^19.0.0",
    "@types/react-dom": "^19.0.0",
    "@types/webextension-polyfill": "^0.12.0",
    "@vitejs/plugin-react": "^4.3.0",
    "chrome-types": "^0.1.0",
    "eslint": "^9.17.0",
    "eslint-config-prettier": "^9.1.0",
    "eslint-plugin-prettier": "^5.2.0",
    "eslint-plugin-react-hooks": "^5.1.0",
    "eslint-plugin-react-refresh": "^0.4.0",
    "husky": "^9.1.0",
    "identity-obj-proxy": "^3.0.0",
    "jest": "^29.7.0",
    "jest-environment-jsdom": "^29.7.0",
    "jest-junit": "^16.0.0",
    "lint-staged": "^15.3.0",
    "prettier": "^3.4.0",
    "ts-jest": "^29.2.0",
    "typescript": "^5.7.0",
    "typescript-eslint": "^8.22.0",
    "wxt": "^0.20.0"
  },
  "lint-staged": {
```

```
    "*.{ts,tsx}": ["eslint --fix", "prettier --write"],
    "*.{json,md,yml}": ["prettier --write"]
  }
}
```

Appendix B: Environment Setup Script

```
#!/bin/bash
# scripts/setup-dev.sh - One-time development environment setup

set -e

echo "Setting up Boba Torrent Extension development environment..."

# Check Node.js version
NODE_VERSION=$(node -v | cut -d'v' -f2 | cut -d'.' -f1)
if [ "$NODE_VERSION" -lt 18 ]; then
  echo "Error: Node.js 18+ required. Current: $(node -v)"
  exit 1
fi

# Install dependencies
echo "Installing dependencies..."
npm install

# Setup git hooks
echo "Setting up git hooks..."
npx husky init

# Create pre-commit hook
cat > .husky/pre-commit << 'EOF'
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"

npx lint-staged
EOF

chmod +x .husky/pre-commit

# Create .env file if not exists
if [ ! -f .env ]; then
  cat > .env << 'EOF'
# Development environment variables
# Copy to .env.local for local overrides
NODE_ENV=development
DEBUG=true
EOF
fi

echo "Setup complete! Run 'npm run dev' to start development."
```

Document generated from 40+ authoritative sources including official Chrome/Mozilla documentation, GitHub repositories, WXT/Extension.js frameworks, and industry best practices.